



Module 04- Assignment

Introduction to programming language with AI (JavaScript + [Node.js](#))

Q1. JavaScript Coding Practice

- Check Weather a number is even or odd
- Find the largest of three numbers
- Reverse a String
- Count Vowel in a string
- Remove Duplicate values from an array

Provide

- Problem Statement
- Source Code
- Output (Add Screen Shot)
- Explanation

Q2. Introduction to Node.Js

Explain

- What is [Node.js](#)
- How does [Node.js](#) works
- Advantage of [Node.js](#) for QA Automation
- Difference between Browser JavaScript and Node.js

Instructor- Mehedi Hasan

Submitted By- Srabony Islam

Date- 10.09.2026

Q1. JavaScript Coding Practice

Introduction

JavaScript is a widely used programming language for developing interactive web applications and server-side applications using Node.js. This assignment demonstrates the use of fundamental JavaScript concepts such as conditional statements, functions, loops, strings, arrays, and built-in methods.

The following programs are implemented and executed using JavaScript.

1. Check whether a number is even or odd.

Problem Statement-

Write a JavaScript program to check whether a number (17) is even or odd.

Source Code-

```
let number = 17;

let remainder = number % 2;

if (remainder === 0) {
    console.log(`${number} is even`);
} else {
    console.log(`${number} is odd`);
}
```

Output-

Problems Output Debug Console Terminal Ports

```
PS C:\Ostad\Assignment Java> node OstadAssignment.js
17 is odd
PS C:\Ostad\Assignment Java> 
```

Explanation-

In this program, the modulus (%) operator is used to find the remainder when the number is divided by 2. If the remainder is equal to 0, the number is even. Otherwise, the number is odd. The `if...else` statement is used to display the appropriate result.

2. Find the largest of three numbers.

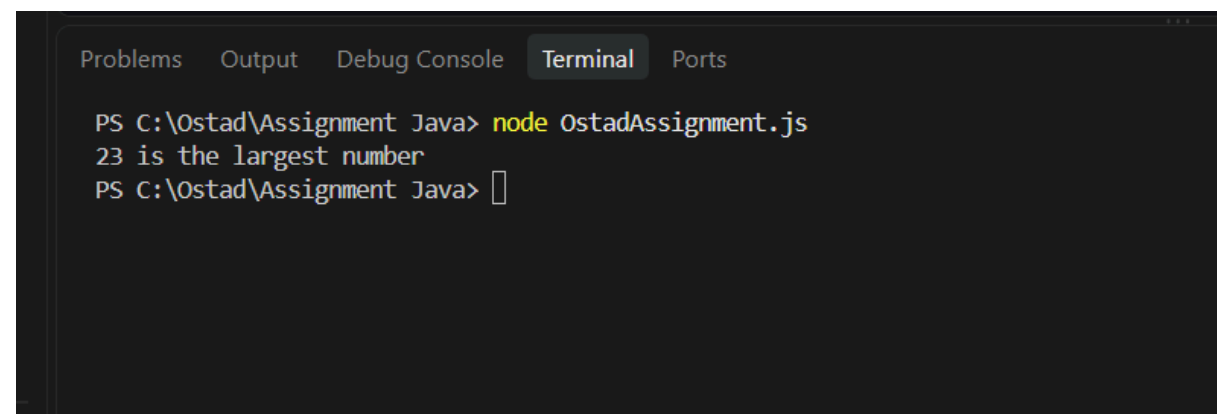
Problem Statement-

Write a JavaScript program to find the largest number among three numbers (16, 11, 23).

Source Code-

```
function findLargest(num1, num2, num3) {  
  
    if (num1 > num2 && num1 > num3) {  
        return `${num1} is the largest number`;  
  
    } else if (num2 > num1 && num2 > num3) {  
        return `${num2} is the largest number`;  
  
    } else {  
        return `${num3} is the largest number`;  
    }  
}  
  
console.log(findLargest(16, 11, 23));
```

Output-



```
Problems  Output  Debug Console  Terminal  Ports  
  
PS C:\Ostad\Assignment Java> node OstadAssignment.js  
23 is the largest number  
PS C:\Ostad\Assignment Java> 
```

Explanation-

This program uses an arrow function named `findLargest()` that accepts three numbers as parameters. The `if...else if...else` statements compare the three numbers. The logical AND (`&&`) operator is used to check whether a number is greater than both of the other numbers. The largest number is then returned and displayed using `console.log()`.

3. Reverse a String.

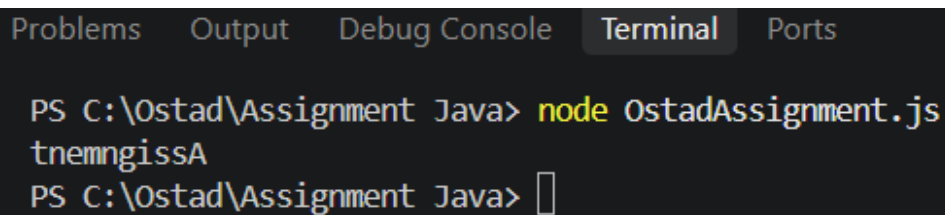
Problem Statement-

Write a JavaScript program to reverse a string (Assignment).

Source Code-

```
function reverseString(str) {  
    return str.split('').reverse().join('');  
}  
  
let string = 'Assignment';  
  
console.log(reverseString(string));
```

Output-



```
Problems  Output  Debug Console  Terminal  Ports  
  
PS C:\Ostad\Assignment Java> node OstadAssignment.js  
tnemngissA  
PS C:\Ostad\Assignment Java> 
```

Explanation-

In this program, the `reverseString()` function takes a string as input and reverses it. The `split('')` method converts the string into an array of individual characters. The `reverse()` method reverses the order of the characters, and the `join('')` method combines them back into a string. Finally, the reversed string is displayed using `console.log()`.

4. Count Vowels in a string .

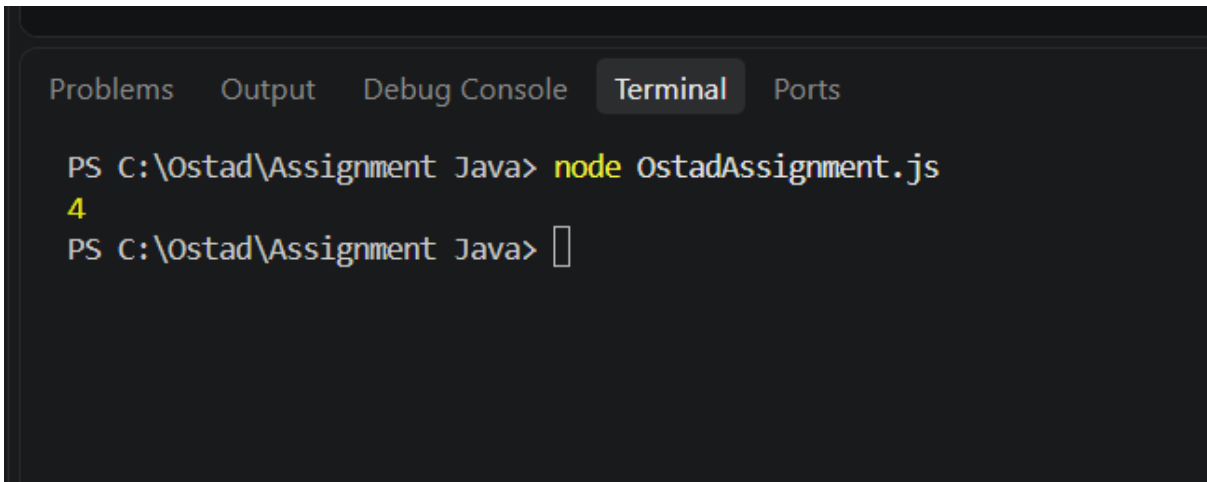
Problem Statement-

Write a JavaScript program to count the number of vowels in a string (Universe)

Source Code-

```
function CountVowels(str) {  
    let count = 0;  
  
    for (const char of str.toLowerCase()) {  
        if (  
            char === 'a' ||  
            char === 'e' ||  
            char === 'i' ||  
            char === 'o' ||  
            char === 'u'  
        ) {  
            count++;  
        }  
    }  
  
    return count;  
}  
  
let string = 'Universe';  
  
console.log(CountVowels(string));
```

Output-

A screenshot of a VS Code terminal window. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (which is active), and 'Ports'. The command prompt shows 'PS C:\Ostad\Assignment Java> node OstadAssignment.js'. The output of the command is the number '4'. The prompt then shows 'PS C:\Ostad\Assignment Java>' with a cursor.

```
PS C:\Ostad\Assignment Java> node OstadAssignment.js
4
PS C:\Ostad\Assignment Java>
```

Explanation-

This program counts the number of vowels in a string. First, `toLowerCase()` converts all characters into lowercase so that both uppercase and lowercase vowels can be counted. A `for...of` loop checks each character of the string. If the character is `a`, `e`, `i`, `o`, or `u`, the `count` variable is increased by one using `count++`. Finally, the total number of vowels is returned and displayed.

5. Remove Duplicate values from an array.

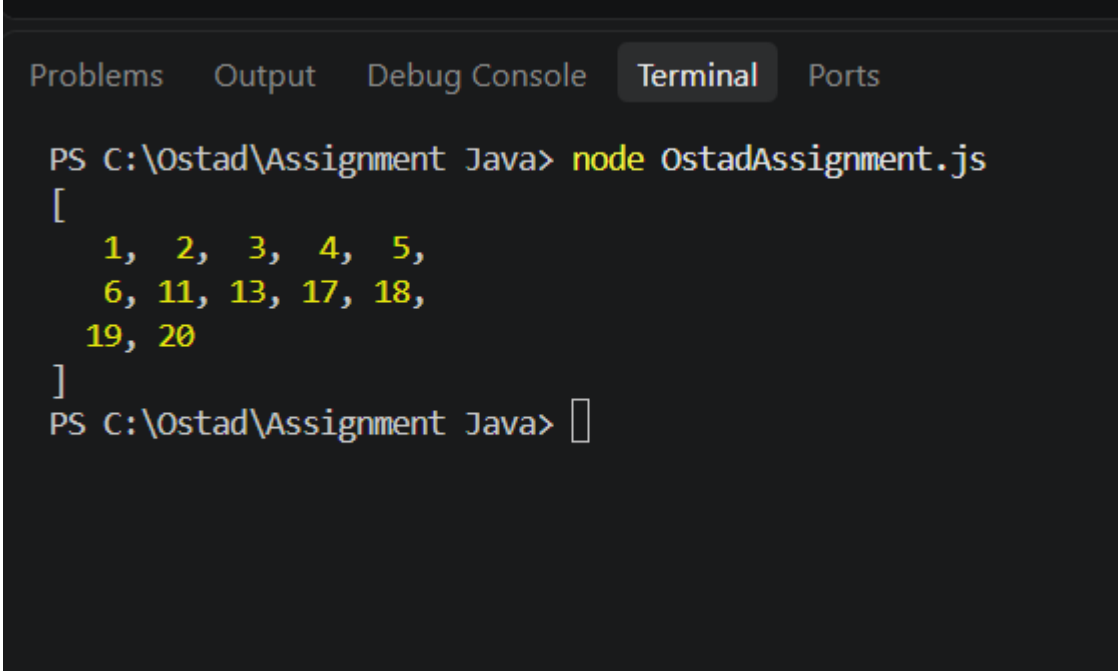
Problem Statement-

Write a JavaScript program to remove duplicate values from an array (1, 2, 3, 4, 4, 5, 5, 6, 11, 13, 13, 17, 18, 19, 20, 20)

Source Code-

```
function removeDuplicates(arr) {
    return [...new Set(arr)];
}
let numbers = [1, 2, 3, 4, 4, 5, 5, 6, 11, 13, 13, 17, 18, 19, 20, 20];
console.log(removeDuplicates(numbers));
```

Output-



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Ostad\Assignment Java> node OstadAssignment.js
[
  1, 2, 3, 4, 5,
  6, 11, 13, 17, 18,
  19, 20
]
PS C:\Ostad\Assignment Java> 
```

Explanation-

This program removes duplicate values from an array using the JavaScript `Set` object. A `Set` automatically stores only unique values. The `new Set(arr)` statement removes duplicate elements from the given array. The spread operator (`...`) is then used to convert the `Set` back into an array. Finally, the new array containing only unique values is displayed using `console.log()`.

Conclusion-

This assignment demonstrates the use of basic JavaScript programming concepts. The programs use conditional statements to make decisions, functions to organize code, loops to process characters, and built-in JavaScript methods to manipulate strings and arrays. Through these programs, fundamental concepts such as operators, functions, loops, strings, arrays, and the `Set` object have been practiced.

Q2. Introduction to Node.Js

1. What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript outside a web browser. It is commonly used to develop the backend or server-side of web applications.

Node.js is built on Google's V8 JavaScript engine and uses an event-driven, non-blocking I/O architecture. This allows applications to handle multiple requests efficiently without waiting for each operation to finish before processing another.

For example, in an e-commerce application, Node.js can be used to handle user login, product searches, shopping carts, orders, and communication between the application and database.

From a Software Quality Assurance (SQA) perspective, Node.js is also relevant because QA engineers may test applications and APIs built with Node.js. They can verify API responses, status codes, authentication, error handling, database integration, performance, and other aspects of software quality.

Therefore, Node.js provides a practical way to use JavaScript for building scalable and efficient server-side applications.

Node.js is popular because it is designed to handle many requests efficiently. It uses a **non-blocking, event-driven architecture**, which makes it particularly suitable for applications that need to handle many simultaneous connections.

Common uses include:

- Web servers and backend applications
- REST APIs
- Real-time applications such as chat systems
- E-commerce applications
- Microservices
- Test automation tools and testing environments
- Command-line tools

2. How Does Node.js Work?

Node.js works as a server-side runtime environment that executes JavaScript code using Google's V8 JavaScript engine. It follows an **event-driven and non-blocking I/O model**, which allows it to handle multiple client requests efficiently.

When a client sends a request to a Node.js application, the request is received by the server and processed according to the required operation. If the operation involves tasks such as accessing a database or reading a file, Node.js can perform these operations without stopping the processing of other requests. Once the required operation is completed, the result is returned to the application, and Node.js sends a response back to the client.

The basic working process can be represented as:

Client Request → Node.js Server → Request Processing → Database/External Service → Response → Client

For example, when a user logs into an e-commerce website, the login request is sent to the Node.js server. Node.js processes the request, communicates with the database to verify the user's information, and then sends a success or error response to the client.

The non-blocking architecture of Node.js allows it to manage many requests efficiently, making it suitable for web applications, APIs, real-time applications, and other systems that require high levels of concurrent activity.

From an **SQA perspective**, understanding how Node.js processes requests is important for testing API functionality, response handling, error management, database integration, concurrency, and application performance.

3. Advantages of Node.js for QA Automation

Node.js provides several advantages for Software Quality Assurance (SQA) and test automation. Since it allows JavaScript to run outside the browser, QA engineers can use JavaScript-based tools and frameworks to automate different types of testing.

1. Fast Test Execution

Node.js uses Google's V8 JavaScript engine and an efficient event-driven architecture, which can support fast execution of automated tests. This is useful when QA teams need to run a large number of test cases.

2. Supports Test Automation Frameworks

Node.js has a large ecosystem of testing tools and frameworks, such as Playwright, Cypress, WebdriverIO, and Jest. These tools can be used for web, API, unit, and end-to-end testing.

3. API Testing

Node.js is well suited for API automation. QA engineers can create automated tests to verify API requests, responses, status codes, authentication, response data, and error handling.

4. Easy Integration with CI/CD

Node.js-based testing tools can be integrated with Continuous Integration and Continuous Delivery (CI/CD) pipelines. This allows automated tests to run whenever new code is committed or deployed, helping teams identify defects earlier.

5. Reusable Test Scripts

Automated test scripts written with Node.js can be organized into reusable functions, modules, and test suites. This reduces duplication and makes test automation easier to maintain.

6. Large Package Ecosystem

Node.js provides access to a large collection of packages through npm (Node Package Manager). QA engineers can use existing packages for testing, reporting, test data generation, API requests, browser automation, and other automation requirements.

7. Supports Parallel Testing

Node.js-based testing frameworks can support parallel test execution. Running multiple independent tests at the same time can reduce overall test execution time and improve the efficiency of the QA process.

8. Useful for Modern Web Applications

Many modern web applications use JavaScript or Node.js technologies. Using Node.js-based automation tools allows QA engineers to work within a technology ecosystem similar to the application they are testing.

Conclusion

Node.js can make QA automation more efficient by providing fast execution, a wide range of testing tools, API testing capabilities, CI/CD integration, reusable test scripts, and support for parallel testing. These advantages help QA teams automate repetitive testing activities, detect defects earlier, and improve overall software quality.

4. Difference Between Browser JavaScript and Node.JS

Browser JavaScript-

Browser JavaScript is JavaScript code that runs inside a web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge. It is mainly used to make web pages **interactive and dynamic**.

For example, browser JavaScript can:

- Respond to button clicks and user actions
- Validate forms before submission
- Change webpage content and styles
- Create interactive menus and features
- Communicate with web servers through APIs

Browser JavaScript can interact with the **Document Object Model (DOM)**, which represents the structure of a webpage. This allows JavaScript to read, modify, or update elements displayed to the user.

For example, when a user clicks the **"Add to Cart"** button on an e-commerce website, browser JavaScript can immediately update the shopping cart displayed on the webpage.

Node.js-

Node.js is an open-source, cross-platform JavaScript runtime environment that allows JavaScript to run outside a web browser. It is commonly used for developing server-side applications, APIs, backend services, and automation tools.

Node.js uses Google's **V8 JavaScript engine** to execute JavaScript code. It also follows an **event-driven and non-blocking I/O model**, which allows it to handle multiple requests efficiently.

For example, in an e-commerce application, Node.js can receive a user's order request, communicate with the database, process the order, and send the result back to the browser.

Node.js is also widely used in **QA automation** because many testing tools and frameworks, such as Playwright, Cypress, and WebdriverIO, use the Node.js ecosystem.

Differences Between Browser JavaScript and Node.js

Although both use JavaScript, they operate in different environments and are designed for different purposes.

Aspect	Browser JavaScript	Node.js
Where it runs	Inside a web browser	Outside the browser
Main purpose	Frontend/client-side development	Backend/server-side development and automation
Webpage access	Can interact directly with the DOM	Does not have a built-in DOM

System access	Restricted for security	Can interact with files and operating-system resources
Common APIs	<code>window</code> , <code>document</code> , browser APIs	File system, HTTP, process, and other Node.js APIs
Package management	Uses browser-based dependencies/build tools	Uses npm to install and manage packages
Common applications	Interactive webpages and user interfaces	APIs, servers, backend services, and automation
QA usage	Mainly supports testing of browser behavior and UI	Widely used for test automation, API testing, and automation frameworks

Summary

The key difference is **where JavaScript runs and what it can access**. Browser JavaScript runs inside the browser and primarily controls the **user interface and webpage behavior**, while Node.js runs outside the browser and provides JavaScript with capabilities needed for **server-side applications, system operations, and automation**.

Therefore, **Node.js does not replace JavaScript; rather, it provides an environment in which JavaScript can be executed outside the browser.**